MODEL X 2.0

Powered By Zuu Crew

PHASE 1 · PROJECT PROPOSAL

Team name Nexio

IEEE Computational Intelligence Society Student Chapter, IIT

Choose one domain

☐	**01**	Cost of living and everyday economics

1. Problem statement and definition

The specific problem VendorPulse addresses is the sudden, localized collapse of daily cash flow for informal street vendors caused by acute external disruptions—such as hyper-local severe weather, spontaneous transit strikes, or unannounced infrastructure failures—which forces them into predatory micro-debt.

This issue primarily affects daily-wage earners, street-level hawkers, and informal market vendors. In urban economic hubs, this demographic represents tens of thousands of individuals who operate entirely on daily liquidity; today's profits fund tomorrow's inventory. When a localized disruption kills daily foot traffic, the cost to these vendors is immediate and severe. Without a buffer, they are forced to borrow working capital from unregulated, predatory local lenders who often charge ruinous interest rates (sometimes upwards of 10% to 20% weekly). The true cost is a rapid descent into a cyclical debt trap, erasing months of financial progress in a single disrupted weekend.

Currently, micro-finance institutions (MFIs) and NGO relief programs address this issue retroactively. They attempt to provide debt restructuring or standard micro-loans only after the vendor has already defaulted or taken on toxic debt to survive the disruption. This approach fails because institutional capital moves too slowly. By the time an NGO identifies a struggling market block, the vendors have already turned to immediate, predatory liquidity just to feed their families and maintain their stalls. Existing systems lack the hyper-local predictive capability required to deploy capital *before* the liquidity crisis forces the vendor's hand.

2. The user

The system is built specifically for a **Micro-finance NGO Field Outreach Coordinator** operating in high-density urban commercial zones.

The primary decision this coordinator makes is resource allocation: deciding exactly which neighborhood or specific market block requires the immediate deployment of rapid, zero-interest emergency bridge loans. They must make these deployment decisions daily. Currently, they make this decision using highly lagging indicators: retroactive default reports, monthly field-officer assessments, or broad, city-wide news of general flooding or strikes. By the time they receive this information, the vendors have already lost their working capital and engaged with predatory lenders.

If VendorPulse exists, the coordinator shifts from a reactive recovery role to a proactive shielding role. Instead of waiting for a vendor to default, the coordinator opens a dashboard on Thursday morning, sees a 72-hour high-confidence liquidity failure alert for a specific three-block radius due to combined road closures and micro-climate storms, and deploys field officers to offer preemptive capital before Friday's trading begins.

3. Literature review

Several approaches attempt to address financial vulnerability in the informal sector, but they fail to provide the predictive, hyper-local intelligence our user requires:

- **Traditional Alternative Credit Scoring Models:** Existing financial inclusion tools focus heavily on establishing alternative credit scores using mobile money transaction histories or utility payments.
    - *What it does:* It predicts general long-term creditworthiness based on historical behavior.
    - *Where it falls short:* It is entirely static and individualized. It cannot account for sudden external environmental or civic shocks that temporarily wipe out cash flow for an otherwise highly reliable vendor.
- **Weather-Indexed Micro-insurance (WII):** Agricultural and rural micro-finance heavily utilizes weather-indexed models.
    - *What it does:* It automatically triggers financial payouts when local sensors detect severe weather anomalies (e.g., rainfall exceeding a set threshold).
    - *Where it falls short:* First, it is strictly retroactive; it pays out only *after* the disaster has struck. Second, it ignores non-weather disruptions such as transit strikes, localized power grid failures, or spontaneous civic unrest, which are equally devastating to urban foot traffic.
- **Macro-Economic Early Warning Systems (EWS):** Large-scale platforms built by international organizations (like the World Bank or IMF) track economic shocks.
    - *What it does:* Predicts broad inflation spikes, currency devaluation, or national supply chain disruptions.
    - *Where it falls short for our user:* The geographic resolution is completely unsuited for local NGOs. A national trend report does not tell an outreach coordinator which specific street intersection will face a liquidity crisis tomorrow.

The Gap: There is a critical void in hyper-local, multi-modal predictive tools. Existing tools are either high-resolution but retroactive (credit scores), or predictive but low-resolution (macro EWS). VendorPulse steps into this gap by fusing localized civic, transit, and environmental data to forecast localized cash-flow collapses before they force vendors into the predatory debt cycle.

4. Proposed solution

- **VendorPulse** is a predictive early-warning intelligence platform that actively monitors overlapping urban data streams—such as hyper-local weather forecasts, transit schedule deviations, and municipal road closure logs—to anticipate when and where street-level foot traffic will abruptly halt. The system continuously processes these disparate data points to evaluate the daily commercial viability of specific urban market blocks, translating complex municipal disruptions into a simple, actionable dashboard for NGO field coordinators.
- The concrete insight the system produces is a localized, 48-to-72-hour cash-flow collapse prediction. Instead of reporting that a storm has happened, it generates a high-confidence risk alert stating, for example, that the intersection of a bus route diversion and incoming heavy rainfall creates a 90% probability of severe liquidity failure for vendors operating in a specific three-block radius by Thursday morning.
- This is an insight the user could not have reached on their own because human operators cannot simultaneously monitor and cross-reference thousands of live municipal, transit, and meteorological data points across a sprawling city. Furthermore, a human coordinator cannot manually calculate the compounded, non-linear effect that multiple minor disruptions (e.g., a closed pedestrian bridge combined with moderate rain) have on hyper-local pedestrian density. The intelligence must be algorithmically extracted from the intersection of these datasets.

5. Data

5.1 Source and access

The VendorPulse system relies on two primary data streams to model hyper-local disruptions:

1. **Meteorological Data:** Hyper-local weather forecasts and historical climate data sourced from the OpenWeatherMap (OWM) One Call API.
2. **Civic and Transit Disruption Data:** Live traffic anomaly data via the Google Maps Traffic API, combined with unstructured local municipal road closure and transit strike announcements scraped from local news RSS feeds (e.g., Ada Derana/NewsFirst APIs) and municipal portals.

Evidence of access: We have successfully registered API keys and executed test queries for both OWM and Google Maps APIs. *(Note to Sahas: Insert a screenshot here showing a successful JSON response from Postman or your terminal pulling weather or traffic data for Colombo).*

Volume and coverage: The APIs provide continuous, real-time coverage mapped across high-density commercial zones (e.g., Colombo 01 through 15). The OWM API allows us to pull granular forecasts up to 8 days in advance, alongside a 40-year historical archive for model training.

Identifiable information: None of our data sources contain Personally Identifiable Information (PII). The system tracks environmental conditions, infrastructural blockages, and broad commercial zone traffic, entirely avoiding individual surveillance or personal financial histories.

5.2 Extraction method

Methodology: We will extract meteorological and traffic data via direct REST API GET requests using Python and standard HTTP libraries. For unstructured municipal announcements and strike warnings, we will deploy automated web scrapers built with Python and Playwright.

Why this method: Direct APIs are the only reliable way to guarantee the structured, machine-readable time-series data necessary for PyTorch forecasting models. However, because municipal authorities often announce road closures or strikes via unstructured press releases, integrating an automated scraper is a necessary supplementary method to capture civic anomalies that APIs miss.

Cleaning and expected issues: We expect the scraped civic data to be highly unstructured, lacking standardized location tags and consistent severity metrics. We will clean this using spaCy (NLP) to extract geographic entities (e.g., "Pettah Market," "Galle Road") and map them to standard coordinates. For the API data, we expect occasional null values or rate-limit timeouts. Our data pipeline will handle this by applying forward-fill imputation for minor gaps and standardizing all timestamps to UTC+5:30 before feeding the tensor arrays.

5.3 Temporality

Data state: VendorPulse utilizes a hybrid temporal approach: a static historical archive is used to train the baseline model, while the production system runs exclusively on **live, streaming data** combined with rolling 72-hour forecasts.

Defense of temporality: Our specific user—an NGO Field Coordinator—must make preemptive cash deployments 48 hours before a crisis hits. A system built on a static historical archive would only prove that weather affects markets generally, which the user already knows. To deliver actionable intelligence, the system must process this week's live incoming storm fronts and today's sudden transit strikes to predict tomorrow's localized cash-flow collapse.

Refresh rate and fallbacks: The live data pipeline refreshes every 3 hours. If a primary source (e.g., the traffic API) becomes unavailable, the system's fallback protocol dictates that it locks onto the last known 24-hour forecast block, temporarily pauses the "live anomaly" scoring, and explicitly flags the dashboard with a "Degraded Confidence: Stale Data" warning so the coordinator knows the exact reliability of the current insight.

6. Approach and methodology

Technical Approach We are implementing a multi-modal time-series forecasting architecture using a Temporal Fusion Transformer (TFT) built in PyTorch, running alongside a Natural Language Processing (NLP) pipeline using spaCy.

The NLP pipeline will ingest unstructured scraped text (news of transit strikes, municipal road closures) and extract the geographic location and severity of the disruption. These extracted entities are then encoded as categorical variables. The TFT model will ingest these variables alongside our continuous numerical data (historical traffic volume, current weather, and 72-hour meteorological forecasts).

This approach fits the problem perfectly because a TFT is explicitly designed to handle a mix of known future inputs (e.g., tomorrow's weather forecast), observed past inputs (e.g., yesterday's traffic density), and static metadata (e.g., the specific coordinates of a market block). Unlike generic models, it allows us to map sudden, non-linear civic shocks directly against continuous environmental data to calculate a highly localized probability of foot-traffic collapse.

Alternatives Considered and Set Aside

1. **ARIMA / Prophet (Traditional Statistical Forecasting):** We initially considered standard time-series models like ARIMA. We set this aside because traditional models assume continuous trends and seasonality. They are highly effective for normal economic forecasting but fail completely at predicting sudden, event-driven "cliff-edges" (like a flash flood or an unannounced bus strike), and they cannot ingest unstructured text data.
2. **Generative LLMs:** We considered using a Large Language Model to synthesize the weather and news data into a risk report. We set this aside because while LLMs excel at parsing the scraped news text, they hallucinate probabilistic risk and lack the rigorous mathematical determinism required for a time-series financial risk engine. The intelligence must come from hard data, not text generation.

Implementation and Testing Workflow Our build process prioritizes data viability over premature modeling:

1. **First (Data Pipeline & Ingestion):** We will build and deploy the automated Playwright scrapers and API integration scripts. We must secure a clean, live data flow before any machine learning begins.
    - *Testing:* We will test the spaCy NER model to ensure it correctly and consistently extracts location tags (e.g., mapping "Pettah" or "Galle Road" to standard coordinates) from unstructured news feeds.
2. **Second (Model Training):** We will train the PyTorch TFT model using the historical OpenWeatherMap and Google Maps archive.
    - *Testing:* We will conduct strict back-testing against known historical disruptions. For example, we will feed the model data leading up to a documented severe Colombo storm or major transit strike in the past two years to verify if the model successfully predicts the anomaly 48 hours prior without being fed the actual outcome.

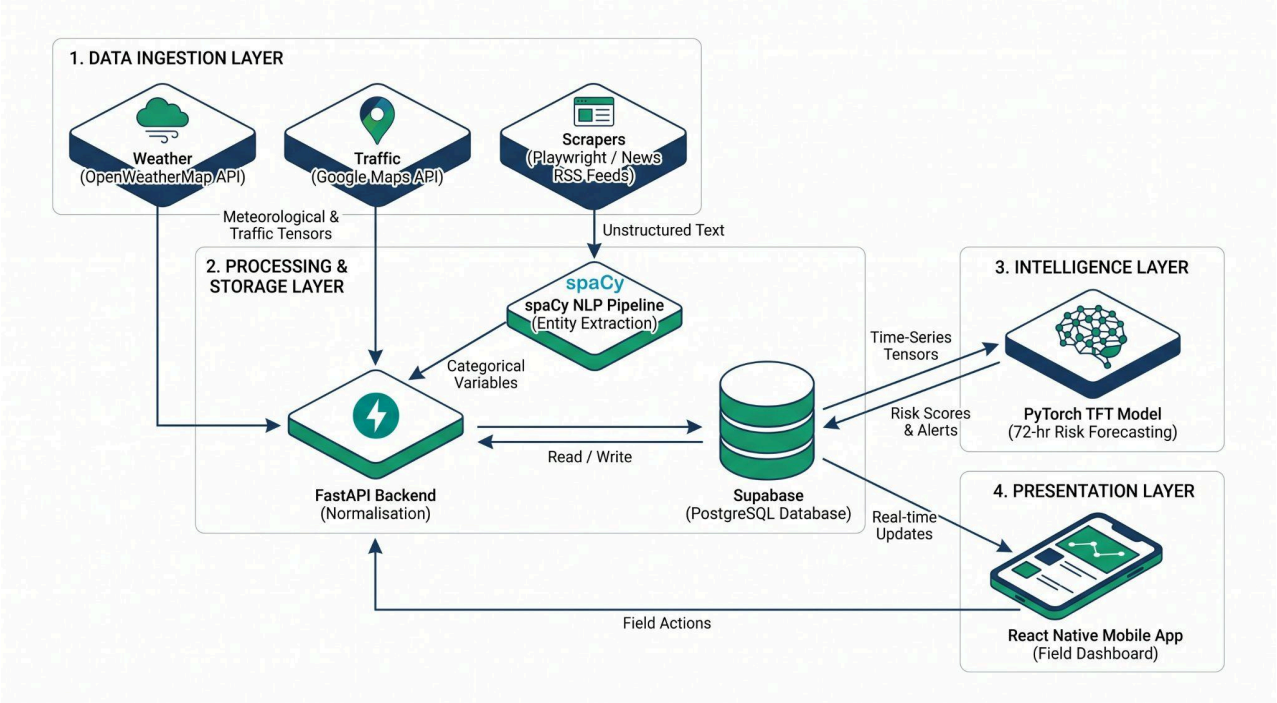
3. **Third (Interface Integration):** Only after the model achieves a viable precision/recall threshold in back-testing will we build the React Native dashboard and wire it to the model's backend endpoint.

7. System design

7.1 Architecture

VendorPulse operates across four primary architectural layers, moving raw municipal data into actionable mobile alerts:

1. **Data Ingestion Layer:** Automated Python workers run scheduled cron jobs every three hours. One set of workers executes REST GET requests to the OpenWeatherMap and Google Maps APIs. Simultaneously, Playwright-driven scrapers pull unstructured transit and municipal announcements from local news RSS feeds.
2. **Processing & Storage Layer:** Raw data lands in a PostgreSQL database via a FastAPI backend. The unstructured text is routed through a spaCy Natural Language Processing (NLP) pipeline to extract geographic entities (Named Entity Recognition) and categorize the severity of the event. All incoming data is normalized into UTC-aligned time-series tensors.
3. **Intelligence Layer:** The processed tensors are fed into the PyTorch Temporal Fusion Transformer (TFT) model. The model calculates a 72-hour forward-looking probability of foot-traffic collapse for predefined commercial zones, writing the risk scores back to the database.
4. **Presentation Layer:** A React Native mobile client queries the FastAPI backend, rendering the risk scores onto an interactive geographic map for the field coordinator.



7.2 Technical stack

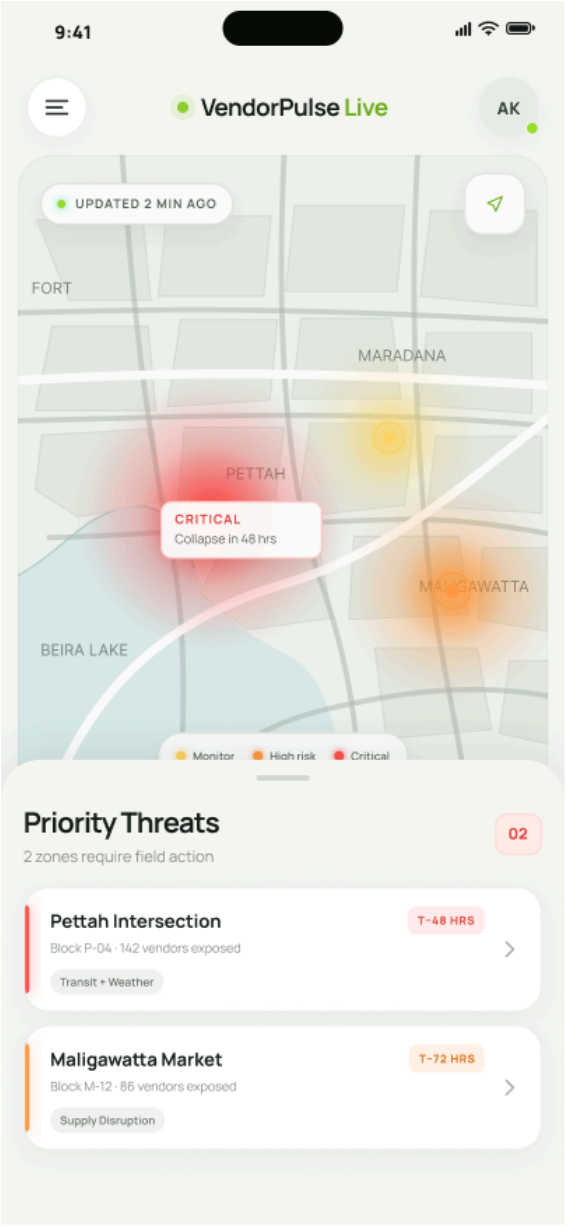
- **Machine Learning & NLP:** PyTorch (for the Temporal Fusion Transformer) and spaCy (for extracting locations from unstructured news). PyTorch is selected because building a custom TFT architecture requires deep low-level tensor control that high-level libraries cannot provide.
- **Backend & API Framework:** Python and FastAPI. FastAPI is chosen for its asynchronous capabilities, making it highly efficient for handling simultaneous I/O bound tasks like API polling and scraping.
- **Database & Infrastructure:** Supabase (managed PostgreSQL). This provides a robust relational database with built-in authentication and real-time subscriptons, allowing the mobile app to receive instant push alerts without complex socket engineering.
- **Frontend / User Interface:** React Native with Tailwind CSS. A mobile framework is essential because our user—an NGO Field Outreach Coordinator—is often mobile and needs on-the-go access rather than being tethered to a desktop web dashboard.
- **Costs and pending access:** The base systems are open-source. However, scaling the Google Maps Traffic API beyond the free tier for high-frequency polling across multiple city blocks will incur minor costs. Cloud hosting for the FastAPI backend and model inference (e.g., Render or AWS EC2) will also require a small deployment budget.

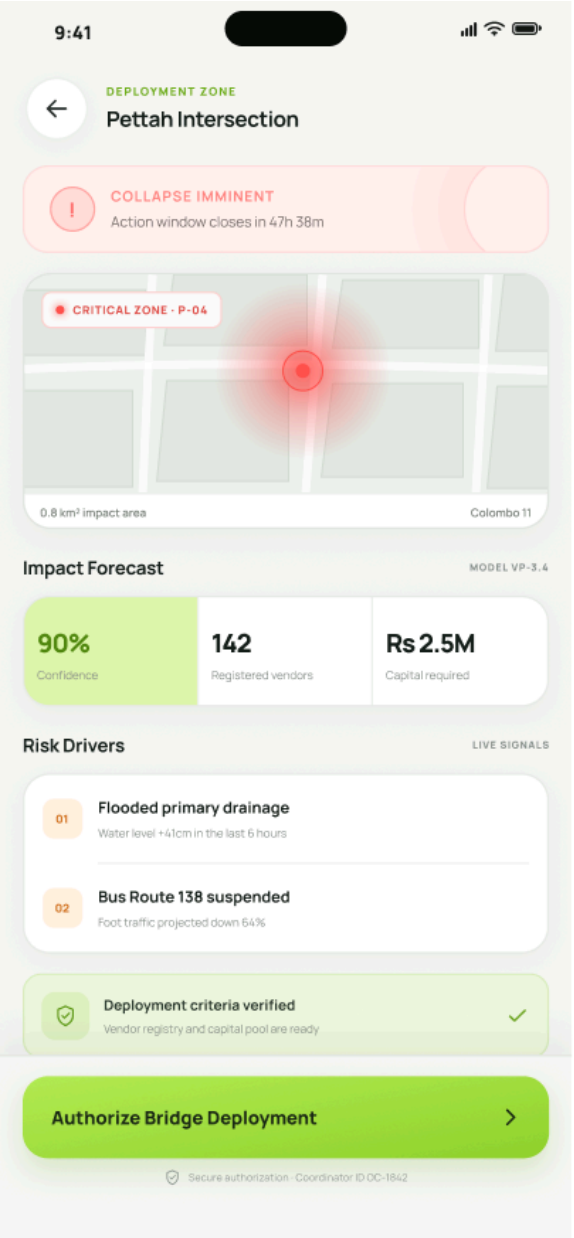
7.3 Interface and interaction

The user reaches the output exclusively through a React Native mobile application designed for high-stress field environments.

What they see: The primary view is a "Threat Map" interface utilizing a dark-mode base with high-contrast, color-coded heat zones (Yellow: Monitor, Orange: High Risk, Red: Critical Liquidity Collapse Imminent). Below the map is a prioritized "Action List" sorted by urgency. Each list item clearly displays the market block name, the predicted time until cash-flow failure (e.g., "T-Minus 48 Hours"), and the primary drivers (e.g., "Flooded intersection + Bus route suspension").

What they do with it: The coordinator taps a specific "Red Zone" on the map. This opens an interaction card detailing the number of registered informal vendors in that block. The coordinator taps a primary action button labeled "Authorize Bridge Deployment." This action instantly logs the decision in the Supabase backend and generates a dispatch alert to local field officers on the ground, directing them to deploy zero-interest working capital to that specific street before the disruption hits.


9:41
VendorPulse Live
AK
UPDATED 2 MIN AGO
FORT
MARADANA
PETTAH
CRITICAL
Collapse in 48 hrs
MALIGAWATTA
BEIRA LAKE
Monitor
High risk
Critical
Priority Threats
2 zones require field action
02
Pettah Intersection
T-48 HRS
Block P-04 · 142 vendors exposed
Transit + Weather
Maligawatta Market
T-72 HRS
Block M-12 · 86 vendors exposed
Supply Disruption


9:41
DEPLOYMENT ZONE
Pettah Intersection
COLLAPSE IMMINENT
Action window closes in 47h 38m
CRITICAL ZONE · P-04
0.8 km² impact area
Colombo 11
Impact Forecast
MODEL VP-3.4
90%
Confidence
142
Registered vendors
Rs 2.5M
Capital required
Risk Drivers
LIVE SIGNALS
01
Flooded primary drainage
Water level +41cm in the last 6 hours
02
Bus Route 138 suspended
Foot traffic projected down 64%
Deployment criteria verified
Vendor registry and capital pool are ready
Authorize Bridge Deployment
Secure authorization · Coordinator ID OC-1842

8. Implementation plan

Build Phase Milestones The implementation window runs from October 20 to November 1. The build is strictly sequenced to ensure data viability before modeling or interface work begins.

Phase	Dates	Milestone	Primary Owner
Data Ingestion	Oct 20 – Oct 22	Establish API connections (Google Maps, OpenWeatherMap), build Playwright scrapers, and configure the Supabase PostgreSQL schema.	Backend/Data Engineer
ML & NLP Engine	Oct 23 – Oct 26	Train and tune the spaCy NLP entity extractor and the PyTorch Temporal Fusion Transformer (TFT) on the historical dataset.	ML Engineer
Integration	Oct 27 – Oct 29	Connect the PyTorch outputs to the FastAPI backend and build the React Native mobile front-end dashboard.	Full-Stack Developer
Testing & Demo	Oct 30 – Oct 31	Run end-to-end live data tests, capture screen recordings, and edit the required 3-minute system demonstration video.	Team Lead / UI Designer
Submission	Nov 1	Final code freeze, complete the technical report, and push the public GitHub repository.	All Members

What "Finished" Looks Like A finished system is a live, automated pipeline where a scheduled cron job pulls current Colombo weather and traffic data, processes it through the PyTorch model, and successfully renders a 72-hour threat map on the React Native

dashboard without manual intervention. The final deliverable is the GitHub repository and the 3-minute video proving this cycle works on real, current data.

Contingency Cuts If we run short of time, we will immediately cut the unstructured NLP scraping pipeline (Playwright and spaCy). We will fall back to relying exclusively on the structured numerical data from the OpenWeatherMap and Google Maps APIs to drive the model, sacrificing some civic anomaly detection to ensure the core predictive pipeline remains functional.

9. Evaluation plan

What "Correct" Means For VendorPulse, "correct" does not mean predicting every minor traffic slowdown or rain shower. Correct means successfully alerting the NGO to a compounding, multi-variable disruption (e.g., severe localized flooding combined with a transit failure) that genuinely halts pedestrian foot traffic in a specific commercial block for a full trading day. The system is correct when it gives the coordinator a mathematically reliable 48-hour window to deploy bridge capital, minimizing false alarms that would waste limited NGO resources.

Measurement and Baselines

Because the Temporal Fusion Transformer (TFT) outputs probabilistic forecasts, we will evaluate the continuous predictions using **Quantile Loss (P50 and P90)** to measure the accuracy of our uncertainty estimates. For the final classification (whether an alert is triggered or not), we will use the **F1-Score**, prioritizing a balance between Precision and Recall to avoid alert fatigue.

We will compare our system's performance against two baselines:

1. **A Naïve Baseline:** Predicting that tomorrow's foot traffic and disruptions will be exactly identical to today's.
2. **A Traditional Statistical Model:** An ARIMA model, which represents the current standard for basic economic time-series forecasting.

Failure Criteria The approach is explicitly not working if the False Positive Rate (FPR) exceeds 20% during out-of-sample testing. In a real-world scenario, an NGO cannot afford to repeatedly send field officers and cash to neighborhoods that do not actually experience a liquidity crisis. Furthermore, if the TFT model fails to significantly outperform the Naïve baseline in accuracy, it indicates that the complex neural network is over-engineered and the data lacks predictive signal.

10. Risks and contingencies

Risk: Unusable unstructured data or scraper failure. The municipal RSS feeds and news sites we intend to scrape change their HTML structures frequently, which could break our Playwright scrapers and render the spaCy NLP pipeline starved for data.

- *Contingency:* If the scraped text proves too chaotic or unreliable by the end of week one, we will abandon the NLP pipeline and rely strictly on the structured numerical data from the Google Maps and OpenWeather APIs. The model will lose some insight into civic strikes, but will remain highly accurate for weather-driven and organic traffic disruptions.

Risk: The Temporal Fusion Transformer (TFT) approach fails to converge. Neural networks require massive, clean datasets. Given our short timeframe, the TFT model might overfit the historical data or fail to outperform the naïve baseline during testing.

- *Contingency:* If the TFT model fails to achieve a viable F1-score during back-testing, we will pivot to a simpler, less data-hungry tree-based ensemble model, such as XGBoost. While XGBoost lacks the deep temporal memory of a TFT, it excels at tabular data classification and is much faster to train and tune under a tight deadline.

Risk: A primary API source disappears or restricts access. Google Maps or OpenWeatherMap could alter their rate limits, making high-frequency live polling too expensive or blocking our server IP.

- *Contingency:* We will implement aggressive request caching in our FastAPI backend so we never query the same endpoint twice in a short window. If live traffic data is completely blocked, we will substitute it with historical time-of-day traffic averages to act as a baseline, shifting the model's weight entirely onto the weather predictions.

Risk: A team member drops out. A sudden loss of a member, particularly the mobile developer or ML engineer, would severely bottleneck the pipeline.

- *Contingency:* All code will be thoroughly documented and pushed daily to a shared GitHub repository. We are containerizing the backend using Docker so any remaining member can run the environment locally. If our frontend developer drops out, we will scrap the React Native mobile app and deploy a rapid, Python-based web dashboard using Streamlit instead, ensuring we still have a working interface for the final demo.

11. References

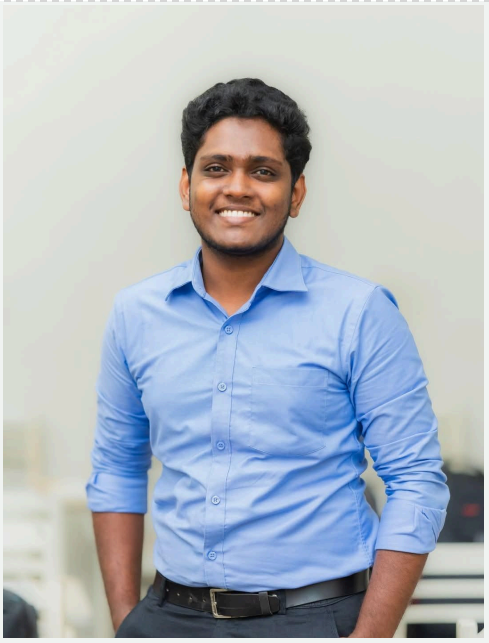
Literature & Methodological Frameworks

- Lim, B., Arık, S. Ö., Loeff, N., & Pfister, T. (2021). "Temporal Fusion Transformers for interpretable multi-horizon time series forecasting." *International Journal of Forecasting*, 37(4), 1748-1764.
- Zhao, et al. (2022). "Impacts and Synergies of Weather Index Insurance and Microcredit in Rural Areas: A Systematic Review." *Environmental Research Letters*.
- Björkegren, D., & Grissen, D. (2018). "Behavior Revealed in Mobile Phone Usage Predicts Credit Repayment." *The World Bank Economic Review*, 32(3), 618-634. (Cited regarding Alternative Credit Scoring).

Software, APIs & Datasets

- OpenWeatherMap. (2026). *One Call API 3.0* [Live weather forecast and historical API]. Retrieved from https://openweathermap.org/api/one-call-3
- Google Developers. (2026). *Google Maps Routes / Traffic API* [Live traffic and routing data]. Retrieved from https://developers.google.com/maps/documentation
- Honnibal, M., Montani, I., Van Landeghem, S., & Boyd, A. (2020). *spaCy: Industrial-strength Natural Language Processing in Python*.
- Paszke, A., et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. In Advances in Neural Information Processing Systems 32 (NeurIPS).

12. Team details



Sahas Abeygunarathne

sahas.samuditha@gmail.com

University of Moratuwa